

S							
I	Paper title	Authors of the paper	Journal	#	Year	Link to the paper	5-7 sentence summary
1	Automatically Characterizing Large Scale Program Behavior	Timothy Sherwood, Erez Perelman, Greg Hamerly, Brad Calder	ACM SIGPLAN Notices	2002		https://cseweb.ucsd.edu/~calder/papers/ASPL05-02-SimPoint.pdf	Goal: Programs exhibit massive variations in behavior over billions of instructions, making. The goal is to automatically identify the large-scale patterns to enable accurate, efficient analysis and simulation. How: Using Basic Block Vectors (BBVs) Definition of BBV: A hardware-independent "fingerprint" that counts how many times each block of code is entered during a fixed interval (e.g., 100M instructions) Comparison: The similarity between intervals is measured using Manhattan distance; if two intervals have similar BBVs, their performance behavior is likely similar. Phase Definition: "phase" is not just a contiguous block of time with no change in a metric, but as any collection of intervals with similar behavior, even if they are far apart in execution . Algorithm: Profile: Collect BBVs for every interval Problem: Processors execute billions of instructions/second; cycle-level view misses large-scale recurring patterns . Goal: Efficient runtime phase detection and prediction to guide optimizations (power, cache, etc.). Key Distinction: Tracks code execution proportions (frequency), not just working sets(old approaches).
2	Phase Tracking and Prediction	Timothy Sherwood, Suleyman Sair, Brad Calder	ACM SIGARCH Computer Architecture News	2003		https://dl.acm.org/doi/pdf/10.1145/871656.859657	Hardware Mechanism: "Fingerprint" based on basic block execution frequency Implementation: Accumulator Table: Hashed Branch PCs increment counters by instruction count. Profiled every 10 million instructions. Classification: "Fingerprint" compared to Past Footprints Table using Manhattan distance. If difference < threshold, classify as existing Phase ID; otherwise create new ID . Overhead: Requires < 500 bytes of on-chip memory. Challenge: Simple predictors fail at phase transitions (stable to unstable). Solution: Run Length Encoding (RLE) Markov Predictor Method: Use RLE set Phase ID, Run Length to index prediction table Methodology: Basic Block Vectors (BBVs)
3	Basic Block Distribution Analysis to Find Periodic Behavior and Simulation Points in Applications	Timothy Sherwood, Erez Perelman, Brad Calder	Conference on Parallel Architectures and Compilation Techniques (PACT)	2001		https://sites.engineering.ucsb.edu/~sherwood/pubs/PACT-bbda.pdf	Metric: Hardware-independent "fingerprint" of code execution for fixed intervals (e.g., 100M instructions) . Dimensions: Each dimension maps to a unique Static Basic Block (single entry/exit code section) in the binary. Values (Weighting): Calculated as (Block Execution Count) *(Number of Instructions in Block); ensures long blocks contribute more to fingerprint than short ones . Normalization: Vector elements divided by total sum; captures proportion of work rather than raw magnitude. Comparison: Manhattan Distance (sum of absolute differences) calculates similarity score (0~2) between intervals. Analysis: Finding Periods & Phases Plots the difference between each interval's BBV and the "Target BBV" (the average of the entire program execution) to visualize how behavior diverges over time Objectives * Extension of serial phase analysis (SimPoint) to shared memory parallel applications (2-4 cores). * Primary Goal: Track phase scalability across varying thread counts (identify which code sections benefit from parallelism). * Secondary Goal: Guide multi-threaded architecture simulation (Parallel SimPoint).
4	Detecting phases in parallel applications on shared memory architectures	Erez Perelman, Marzia Polito, Jean-Yves Bouguet, John Sampson, Brad Calder, Carole Dulong	IPDPS'06: Proceedings of the 20th international conference on Parallel and distributed processing	2006		https://dl.acm.org/doi/10.5555/1898953.1899021	Hardware & Validation Metrics * Platform: 4-node Intel Itanium II (private L2/L3 caches, shared main memory, 1 thread/node). * Benchmarks: OpenMP; p-threads, NAS Parallel Benchmarks (NPB), bioinformatics (SNP, SVM RFE). * Validation Metric: Standard deviation (variance) of hardware metrics (CPI, L2/L3 hit rates). * Success Condition: Intra-phase variance significantly lower than total execution variance. Profiling & Data Collection * Tool: Intel VTune Performance Analyzer (runs on native hardware, <2% overhead). * Metric: Extended Instruction Pointers (EIPs) sampled once every 100k instructions. * Vector Creation: EIP Vectors (EIPVs) track sampled EIP frequencies per interval. * EIPs mapped back to static code blocks via binary analysis (Pin). * Result: Sampled Basic Block Vectors (BBVs) (filters out sampling noise). Algorithm: Single Parallel Run * Flawed Approach: Agglomerating all thread data (destroys per-thread visibility/inconsistent). * Chosen Approach: Independent per-thread tracking.

S	I	Paper title	Authors of the paper	Journal	#	Year	Link to the paper	5-7 sentence summary
	5	P4: Phase-based power/performance prediction of heterogeneous systems via neural networks	Yeseong Kim; Pietro Mercati; Ankit More; Emily Shriver; Tajana Rosing	2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)		2017	https://ieeexplore.ieee.org/document/8203843	Objectives * Target: Cross-platform power and performance prediction for workload migration in heterogeneous systems (servers, mobile, IoT). * The Problem: Traditional PMC-based linear models failing across different CPU architectures (e.g., x86 vs. ARM). Source-code instrumentation causing unacceptable overhead. * Proposed Solution: framework. Dynamic identification of machine-independent phases using "only*" runtime hardware PMCs and Neural Networks (no source code required). Single-Machine Characterization (Offline) * Objective: Construction of baseline power models and local phase identification per machine. * Automated Event Selection (Lasso): Utilization of l1-regularization (Lasso) to filter out weakly correlated PMCs. * Elimination of manual, domain-expert selection. * Isolation of ~11-12 highly power-correlated PMCs (e.g., L1/LLC misses, branch misses). Phase Extraction (Clustering): * Application of k-means clustering to group PMC vectors into distinct "application phases." * kNeedle Algorithm: Automated selection of optimal cluster count () by finding the "knee" (convergence point) in the standard deviation curve. * k-means++ Used for optimal initial centroid placement Core Objectives * Run-time phase detection and classification without source code/recompilation. * Primary goal: Estimate per-phase L2 cache demand to optimize utilization. * Method: Track L2 cache access patterns (hybrid of Working Set Signatures and Basic Block Vectors).
	6	A Run-Time Program Phase Detection Technique for Optimizing Per-Phase L2 Cache Demand	Ibrahim E. Ziedan, Hazem I. Shehata and Shaymaa M. Serag	The Egyptian International Journal of Engineering Sciences and Technology		2016	https://www.semanticscholar.org/reader/5bb6aed259699b3cbd192bcfd270eec1e327e41f	Data Collection (Hardware-Level) * Tool: Intel Performance Monitoring Unit (PMU) via Linux 'perf_events'. * Hardware Features: Precise Event-Based Sampling (PEBS) & Data Linear Address (DLA). * Tracked Event: 'MEM_LOAD_UOPS_RETIRED.L1_MISS'. * Mechanism: When an L1 data miss occurs (triggering an L2 access), the PMU captures the specific memory address. Vector Construction: Cache Access Signature Vector (CASV) * Structure: 1D array; total elements () = number of sets in the L2 cache. * Mapping: Captured memory addresses are hashed to their corresponding L2 cache set. * Value: The array element (a n-bit saturating counter) increments, capturing both *which* sets are accessed and the *access intensity*.
	7	Learning-based Phase-aware Multi-core CPU Workload Forecasting	ESA Lozano, A Gerstlauer	ACM Transactions on Design Automation of Electronic Systems		2022	https://dl.acm.org/doi/pdf/10.1145/3564929	Phase Classification Algorithm Goal: Predict multi-core CPU workload metrics (e.g., Cycles Per Instruction - CPI) for dynamic resource optimization (DVFS, thermal management, cache tuning). Problem: Workloads have abrupt "phase" changes. Single "phase-unaware" AI models fail during these shifts. Gap: Prior work ignores multi-core interference (shared resource contention). Proposed Solution: 3-Stage Phase-Aware Pipeline Stage 1: Phase Classification & Separation * Groups hardware counter data into distinct phases (unsupervised). * Splits chaotic time-series into isolated, smooth "phase buckets". Stage 2: Phase Prediction * Predicts future phases using historical data. Stage 3: Phase-Based Forecasting * Uses highly specialized models (one per phase) to predict exact CPI values. * Uses Stage 2 predictions to select which model's output to use. Multi-Core Architectures Evaluated Objectives Target: Energy-efficient frequency scaling on heterogeneous multi-GPU systems (AMD, Intel, NVIDIA). The Problem: Coarse-grained DVFS (one frequency per application) ignores per-kernel energy diversity; fine-grained DVFS (one frequency per kernel) incurs costly frequency-change overhead (0.30–0.60 ms per change), which can negate energy savings — especially for applications with many lightweight kernels in loops. Proposed Solution: A phase-based approach that groups kernels with similar energy characteristics into "phases," sets one optimal frequency per phase, and hides frequency-change overhead by overlapping it with MPI communication in multi-GPU programs. Methodology Application modeled as an Energy-aware DAG extracted from SYCL task graphs: vertices = kernels, edges = dependencies, annotated with time/energy across all frequencies. One-shot profiling at default frequency + SYnergy energy prediction model eliminates the need to run the application at every frequency. Loop repetition counts stored as metadata to avoid DAG explosion. Three phase detection algorithms compared: Dynamic Programming (optimal, $O(T^4)$), Greedy ($O(T^3)$), and Clustering ($O(T^3)$). Clustering selected — achieves 83%–100% of optimal energy savings with tractable complexity. Cost Function balances: frequency-change overhead, task execution time at the chosen phase frequency, and loop repetition factors.
	8	Phase-based Frequency Scaling for Energy-efficient Heterogeneous Computing	Lorenzo Carpentieri, Antonio De Caro, Majid Salimi Beni, Kaijie Fan, and Biagio Cosenza	2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)		2025	https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=11078489	Objectives Target: Energy-efficient frequency scaling on heterogeneous multi-GPU systems (AMD, Intel, NVIDIA). The Problem: Coarse-grained DVFS (one frequency per application) ignores per-kernel energy diversity; fine-grained DVFS (one frequency per kernel) incurs costly frequency-change overhead (0.30–0.60 ms per change), which can negate energy savings — especially for applications with many lightweight kernels in loops. Proposed Solution: A phase-based approach that groups kernels with similar energy characteristics into "phases," sets one optimal frequency per phase, and hides frequency-change overhead by overlapping it with MPI communication in multi-GPU programs. Methodology Application modeled as an Energy-aware DAG extracted from SYCL task graphs: vertices = kernels, edges = dependencies, annotated with time/energy across all frequencies. One-shot profiling at default frequency + SYnergy energy prediction model eliminates the need to run the application at every frequency. Loop repetition counts stored as metadata to avoid DAG explosion. Three phase detection algorithms compared: Dynamic Programming (optimal, $O(T^4)$), Greedy ($O(T^3)$), and Clustering ($O(T^3)$). Clustering selected — achieves 83%–100% of optimal energy savings with tractable complexity. Cost Function balances: frequency-change overhead, task execution time at the chosen phase frequency, and loop repetition factors.

S	I	Paper title	Authors of the paper	Journal	#	Year	Link to the paper	5-7 sentence summary
	9	Sampling-Based Phase Classification and Prediction for Multi-threaded Program Execution on Multi-core Architectures	Chin-Hao Chang; Pangfeng Liu; Jan-Jan Wu	International Conference on Parallel Processing (ICPP)		2013	https://ieeexplore.ieee.org/document/6687368	Objectives Target: Online phase classification and next-phase prediction for multi-threaded parallel programs on multi-core architectures. The Problem: Prior phase detection work mostly focused on single-threaded programs using full code signatures collected on simulators – too expensive for real-time use with parallel programs on real hardware. Proposed Solution: A lightweight sampling-based framework that collects code signatures per thread at runtime, classifies intervals into phases using a shared centroid table, and predicts the next phase using Markov-based methods – all with under 3KB of memory. Phase Classification Framework Code Signature: Sampled Frequency Vector – a 32-dimensional vector built by sampling one Extended Instruction Pointer (EIP) per 1M instructions retired, right-shifting the EIP by 8 bits to approximate its basic block region, hashing to one of 32 counters. One interval = 100M instructions (100 samples). Classification: Manhattan distance between the current interval's vector and all centroid vectors in a shared Centroid Table. If minimum distance < similarity threshold (set to 61), classify into that phase and update centroid; otherwise create a new phase. LRU replacement when phases exceed 64 entries. Shared Centroid Table: All threads share a single table, enabling cross-thread identification of intervals executing the same code sections with the same phase IDs. Confidence Table: Acts as a noise filter – new groups must accumulate a minimum number of intervals (Min Counter threshold) before being promoted to the centroid table as a real phase. Groups below the threshold are labeled "Transition Phase" (phase ID 0). Dramatically reduces spurious phases and improves prediction accuracy.
	10	Predicting Program Phases and Defending Against Side-Channel Attacks using Hardware Performance Counters	Junaid Nomani, Jakub Szefer	HASP '15: Proceedings of the Fourth Workshop on Hardware and Architectural Support for Security and Privacy		2015	https://dl.acm.org/doi/pdf/10.1145/2768566.2768575	Objectives Target: Mitigating microarchitectural side-channel attacks through security-aware OS scheduling, with secondary performance benefits from reduced functional unit contention. The Problem: Applications sharing the same core share functional units (caches, FPUs, branch predictors), creating side-channel attack surfaces. Current schedulers are blind to what hardware resources applications are using and cannot prevent dangerous co-scheduling. Proposed Solution: Augment the Linux scheduler with hardware performance counter (HPC) monitoring and a neural network predictor that forecasts upcoming program phases, enabling proactive scheduling of applications with dissimilar resource demands onto the same core. System Architecture Three components: (1) Modified kernel scheduler that resets/reads HPCs at each context switch and stores values in the task structure; (2) PMC kernel module that asynchronously relays counter data; (3) User-space ML module that runs the neural network and returns predictions. Predictions written back into the task structure so the scheduler can use them for core placement decisions. Sits on top of (does not replace) Linux CFS for time-slicing. Machine Learning Approach 7-layer feed-forward neural network (5 hidden layers), trained offline per program on full SPEC2006 runs. Input: last 15 context switches of counter data, simplified to 5 k-means clusters per counter (centroids unique per program). Input layer = 75 neurons, output = 5
	11	Multilevel Phase Analysis	Weihua Zhang, Jiaxin Li, Yi Li, Haibo Chen	ACM Transactions on Embedded Computing Systems (TECS)			https://dl.acm.org/doi/epdf/10.1145/2629594	
	11	Program phase detection and exploitation	Chen Ding, S. Dwarkadas, M.C. Huang, Kai Shen, J.B. Carter	IEEE International Parallel & Distributed Processing Symposium		2006	https://ieeexplore.ieee.org/document/1639570	
	12	SimPoint 3.0: Faster and More Flexible Program Phase Analysis	Greg Hamerly, Erez Perelman, Jeremy Lau, Brad Calder	Journal of Instruction-Level Parallelism		2005	https://cseweb.ucsd.edu/~calder/papers/JILP-05-SimPoint3.pdf	